

ARM PrimeCell

DMA Controller (PL081)

Integration Manual

ARM PrimeCell DMA Controller (PL081) Integration Manual

© Copyright ARM Limited 2001. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell DMA Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	7
4.4	Customisation	7
4.4.1	DMA Channels	8
4.4.2	FIFO Sizes	8
4.4.3	AHB Master Ports	8
4.4.4	DMA Requestors	8
4.4.5	AHB Master Bus Width	9
4.4.6	AHBLite Master Ports	9
4.5	Synthesis	9
4.5.1	Setup Scripts	10
4.5.2	Command Script	10
4.5.3	Constraint Scripts	10
4.6	Integration with AMBA Micropack components	11
4.7	Interrupt Connection	11
4.8	Clocking	12
4.9	Clock Tree synthesis	12
4.10	Resets	12
4.11	Reset Buffering	12
4.12	Synchronisers	12
4.13	Endianness	12
4.14	Integration Testing	13

4.15	Production Testing	13
4.15.1	Scan Insertion and ATPG	13
4.16	Functional and Timing Verification	14
5	APPENDIX A	14
5.1	PrimeCell Environment Variables	14

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A01	27 April, 2001	First draft
A	17 May, 2001	First release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0218A	ARM PrimeCell DMA Controller (PL081) Technical Reference Manual
3	PL081 DDES 0000	ARM PrimeCell DMA Controller (PL081) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05
7	SCB00 QPRO 0008 A05	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
DMA	Direct Memory Access
DMAC	DMA Controller

2 SCOPE

This document describes how the ARM PrimeCell DMA Controller (DMAC PL081) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell DMA Controller is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell DMA Controller (PL081) Technical Reference Manual [Ref.2].

The ARM PrimeCell DMA Controller (PL081) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell DMA Controller block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

The PrimeCell Integration Vector Strategy document [Ref. 7] describes the motivation and use of integration test vectors.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell DMA Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell DMA Controller block.

An example ASIC design flow is described below:

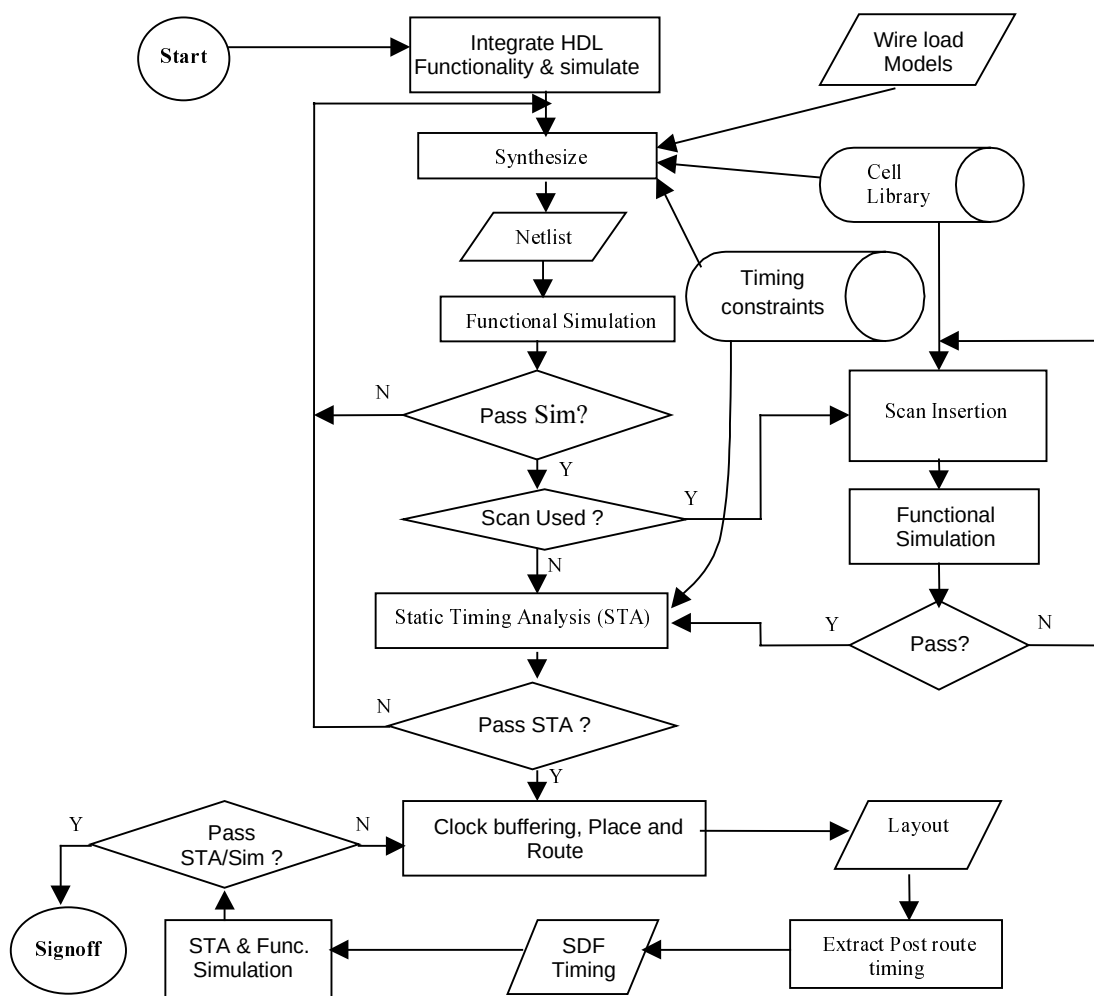


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell DMA Controller block. For further information refer to the ARM PrimeCell DMA Controller (PL081) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus. This is the only active LOW signal.
HADDR[11:2] Address bus	Input	AHB Master	The system address bus.
HWRITE Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZE[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer. The DMAC supports only 32 bit WORD accesses to all its registers. So all transfers to and from the DMA controller must be 32-bit (HSIZE[2:0] = '010').
HTRANS Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer. HTRANS[1] on the AHB bus must be connected to the single-bit-wide HTRANS input of the DMAC. '1' represents NSEQ or SEQ and thus a valid data transfer. '0' represents BUSY or IDLE.
HWDATA[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the DMAC Slave Interface during write operations. The DMAC Slave supports only 32-bit accesses
HSELDMAC Slave select	Input	AHB Decoder	The DMA controller AHB slave has its own slave select signal. This signal indicates that the current transfer is intended for the DMAC. This signal is simply a combinatorial decoding of the address bus.
HRDATA[31:0] Read data bus	Output	AHB Master	The read data bus is used to transfer data from DMAC Slave Interface to the bus master during read operations. The DMAC Slave supports only 32-bit accesses
HREADYIN Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus and the DMAC Slave can starts its own transfer.
HREADYOUT Transfer done	Output	AHB Master and Slave	When HIGH the HREADYOUT signal indicates that a transfer has finished on the bus from the DMAC Slave. The DMAC AHB Slave interface inserts one wait state for read accesses to Channel registers. All other read/write accesses to registers are completed with zero wait states.

HRESP[1:0] Transfer response	Output	AHB Master and Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The DMA Controller AHB Slave Interface only drives OKAY or ERROR response.
---------------------------------	--------	----------------------	---

Table 4.1-1 AHB slave signals

Signal Name	Type (Input/Output)	Source / Destination	Description
HADDRM[31:0] Address bus	Output	AHB Slave	The 32-bit system address bus.
HTRANSM [1:0] Transfer type	Output	AHB Slave	Indicates the type of the current transfer, which can be NONSEQ, SEQ, IDLE or BUSY.
HWRITEM Transfer direction	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZEM [2:0] Transfer size	Output	AHB Slave	Indicates the size of the transfer. The DMA controller allows 8, 16 and 32-bit transfer sizes. 8-bit: HSIZEM[2:0] = '000' 16-bit: HSIZEM[2:0] = '001' 32-bit: HSIZEM[2:0] = '010'
HPROTM [3:0] Protection control	Output	AHB Slave	Provides additional protection information about a bus access. These signals are intended for use by any module that wishes to implement some level of protection based on - whether the transfer is a privileged mode access or user mode access - whether the current access is cacheable or bufferable.
HLOCKDMAC Locking information for locked transfers	Output	AHB Arbiter	This signal is used to indicate to the arbiter that the requesting master is going to perform a number of indivisible transfers. The arbiter must not grant the bus to another bus master once the locked transfer has commenced.
HBURSTM [2:0]	Output	AHB Slave	Indicates the size of the burst for burst transfer. The types of bursts supported by the DMAC AHB Master are INCR, INCR4, INCR8 and INCR16.
HWDATAM [31:0] Write data bus	Output	AHB Slave	The write data bus is used to transfer data from the master to the bus slaves during write operations.
HRDATAM [31:0] Read data bus	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during read operations.
HREADYINM	Input	AHB Slave	When HIGH the HREADYINM signal indicates that a transfer

Transfer done			has finished on the bus. This signal may be driven LOW to extend a transfer, by the AHB slave.
HRESPM [1:0] Transfer response	Input	AHB Slave	The transfer response provides additional information on the status of a transfer. Four different responses supported are OKAY, ERROR, RETRY and SPLIT. To indicate that the transfer has completed successfully, the slave uses the OKAY response. To indicate that an error has occurred, the slave uses the ERROR response. The DMA controller will then assert the Error Interrupt request (DMACINTERR). The slave indicates that the data is not ready by using the SPLIT and RETRY responses. The DMA controller should retry the transfer at a later date.

Table 4.1.2 AHB Master signals

Signal Name	Type (Input/Output)	Source / Destination	Description
HBUSREQDMAC	Output	AHB arbiter	Bus request signal used by the DMA controller to request the AHB bus.
HGRANTDMAC	Input	AHB arbiter	This signal is used to indicate that the DMAC master is selected. The master gains bus ownership when HGRANTDMAC and HREADYINM are high on the rising edge of HCLK.

Table 4.1.3 AHB Master bus request signals

Signal Name	Type (Input/Output)	Source / Destination	Description
DMACINTERR	Output	Interrupt Controller	DMAC Error interrupt request to interrupt controller. Note that this interrupt signal is active HIGH.
DMACINTTC	Output	Interrupt controller	DMAC Count interrupt request to interrupt controller. Note that this interrupt signal is active HIGH.
DMACINTR	Output	Interrupt controller	DMAC request to interrupt controller. This signal is the combination of DMACINTERR and DMACINTTC. Note that this interrupt signal is active HIGH.

Table 4.1.4 DMAC Interrupt signals

Signal Name	Type (Input/Output)	Source / Destination	Description
DMACBREQ[15:0]	Input	Peripheral	DMAC Burst transfer request.
DMACSREQ[15:0]	Input	Peripheral	DMAC Single transfer request.
DMACLBREQ[15:0]	Input	Peripheral	DMAC Last burst transfer request.
DMACLSREQ[15:0]	Input	Peripheral	DMAC Last single transfer request.
DMACCLR[15:0]	Output	Peripheral	DMA Request Acknowledge/Clear. It is asserted one clock after the end of the data-phase of the last beat of the transfer.
DMACTC[15:0]	Output	Peripheral	DMAC Terminal Count. Indicates that the transaction has completed and the packet of data has been transferred.

Table 4.1.5 DMAC request and response signals

Signal Name	Type (Input/Output)	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	Scan data output for observing the flip-flop contents in scan test mode.

Table 4.1.6 Scan Test related signals

4.3 Bus Interconnection

The DMAC block interfaces with the AMBA AHB interface using separate uni-directional read and write data buses HRDATA and HWDATA. For further information refer to the AMBA AHB Specification [Ref.1].

The data buses from the DMAC should be interfaced with other data buses from other peripherals using multiplexers at the chip level.

However, it is also possible to interface the DMAC block with a 3-state bi-directional data bus. A wrapper needs to be added around the DMAC block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the AHB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Customisation

The ARM PrimeCell DMA Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional

test program and the testbenches where required. Synthesis will have to be rerun to ensure that timings are met. Information to support such modification is included in the ARM PrimeCell DMA Controller (PL081) Design Manual [Ref.3].

4.4.1 DMA Channels

The ARM PrimeCell DMAC (PL081) implements 2 DMA channels. If the number of channels need to be changed, the following changes will need to be done :

- The channel block is generic and the same block (DmacChannel) may be instantiated 'N' times in the top-level of the Dmac, where N stands for the number of channels. Every channel takes all the request lines and identifies the pertinent request within itself.
- The number of ports entering the DmacRspRoute block will change with the change in number of channels. The number of ReqTCx and ReqClr signals as an input to DmacRspRoute will increase/decrease, though the width of each ReqTC/Clr signal will stay the same ((number of requestors – 1) down to 0). DmacRspRoute does a logical OR of all these TC/Clr ports and sends out the DMACTC and DMACCLR vectors.
- The channel-specific inputs to the DmacRspRoute, i.e. the raw-interrupt and masked-interrupt lines will also change in number, but the operations to be done on these i.e. concatenation for status-register-read purposes and ORing for DMACINT* lines, remain unchanged.
- The Internal Arbiter (DmacArbiter) will have to arbitrate between a different number of channels with each configuration. So, the Channel-Internal Arbiter interface will change, but the Internal Arbiter – Master Interface connection will remain the same.

4.4.2 FIFO Sizes

The FIFO sizes in all the channels are defined in the HDL source code to be 32-bit wide and 4 locations deep. These sizes should be adequate for most systems. For optimality, it is recommended that the FIFO width should be the same as the data bus width. If different size FIFO buffers are required, the FIFO register file and the Data packing-unpacking logic (DmacChRegFile and DmacChPckUnpck) can be modified accordingly. In addition, certain other blocks also need to be modified since there is bound to be an impact on the number of AHB transfers that can be posted by the Channel to the AHB Master. In particular, the signals that require attention are the *NumOfXfers* signals originating from the Source/Destination transfer control blocks (DmacChSrcXfer and DmacChDstXfer) and propagating through other blocks (DmacChReqProc and DmacChannel) before terminating at the AHB Master block (DmacAhbMaster). If this is done, the functional test program must also be modified.

4.4.3 AHB Master Ports

The ARM PrimeCell DMAC (PL081) implements single AHB Master ports. If the number of AHB Master ports need to be increased, the following changes will need to be considered :

- The muxes for signals entering the DmacChannel blocks from the DmacAhbMaster, need to be made N-to-1 muxes, where N stands for the number of AHB Master instances.
- Demuxes for signals from the DmacChannel blocks to the DmacAhbMaster instances need to be changed to account for the increased number of AHB Master instances.
- The modules affected are DmacChLLILoad, DmacChDstXfer, DmacChPckUnpck, DmacChRegBlock, DmacChReqMask, DmacChReqProc, DmacChSrcXfer, DmacChannel, DmacAhbSlaveIf and Dmac.

If this is done, the functional test program must also be modified.

4.4.4 DMA Requestors

The ARM PrimeCell DMAC (PL081) supports 16 DMA requestors. If the number of DMA requestors need to be changed, software needs to take care when programming the Source-Peripheral-Select and Destination-

Peripheral-Select register fields (in DMACCxConfig register). For example, if a 8-requestor configuration is used, then software should not program any value greater than 7 in the Peripheral-select bit-field, even though the internal register fields are implemented using 5 flip-flops which can hold values upto 31.

Additionally, DMA request multiplexers in the DmacChReqProc block will need to be modified to match the number of requestors.

4.4.5 AHB Master Bus Width

The AHB Master ports in the ARM PrimeCell DMAC (PL081) have 32-bits-wide data buses. If the width of the data bus needs to be increased, the following changes will need to be considered :

FIFO Size

For optimality, it is recommended that the FIFO width should be the same as the data bus width. Thus with change in data bus width, the FIFO width will need to change. The changes related to this are outlined in **Section 4.4.2 FIFO Sizes**.

Endianization logic

The endianization logic in the DmacLiteMaster block currently assumes a 32-bit wide data bus width. This logic will change in case of AHB Master Bus Width change.

AHB Master Interfaces

The widths of ports on the AHB Master Interfaces will change with change in the widths of the HWDATA and HRDATA lines connected to the interfaces.

4.4.6 AHBLite Master Ports

Each AHB Master port in the ARM PrimeCell DMAC (PL080) is implemented by interconnecting an AHBLite Master and an AHBLite-to-AHB wrapper. If the user intends to use the DMAC in an AHBLite system, then the AHBLite-to-AHB wrapper can be eliminated, to allow the AHBLite Master in the DMAC to directly interface to the AHBLite bus. The following changes will need to be done in this case :

- The DmacMasterWrap block needs to be removed.
- All AHBLite signals from the DmacLiteMaster block need to be brought out as ports of the Dmac top level. These can then be connected to the respective AHBLite bus signals.
- The MREADY input to the Dmac (and DmacLiteMaster) needs to be driven by the HREADY signal on the AHBLite bus.
- The MERROR input to the Dmac (and DmacLiteMaster) needs to be driven by bit 0 of the HRESP signal on the AHBLite bus. Note that Split/Retry responses are not defined for AHBLite devices.

4.5 Synthesis

The DMA Controller block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Dmac.cmd - synthesizes the DMA Controller to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Dmac_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.5.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

4.5.2 Command Script

The master compile command script Dmac.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Dmac.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Dmac_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.5.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file equivalent to the library_avanti_cb25_v2.1.scr file.
- b) Timing constraints for AHB bus signals are set via the amba_params.scr, ahb_master.scr and the ahb_slave.scr files. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
 - the plus uncertainty value is set at 40% of the minus uncertainty value.
- The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume the following bus timing budgets :

- 30 % input setup on all input ports including those on the AHB Slave port, the AHB Master ports and the DMA request lines.
 - 40 % output valid delay on all outputs including those on the AHB Slave port, the AHB Master ports and the DMA response lines.
- c) Timing constraints for non-AMBA DMAC-specific signals are set via the Dmac.scr file. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.
- d) Timing exceptions are specified via the Dmac_exceptions.scr file. False paths are specified between non-AMBA outputs (DMACCLR, DMACTC, DMACINTERR and DMACINTTC) and the AHB slave read paths through HRDATA, as these paths are expected to be exercised only during integration testing and not in normal mode of operation. It is not envisaged that Integration tests will be required to be run at full speed (100 MHz).

4.6 Integration with AMBA Micropack components

The DMA Controller block is designed to be integrated into an AMBA system via the AHB (Advanced High-Performance Bus) slave port and the two AHB Master ports. It is designed to work in systems where the DMAC, through its AHB Master ports, can read from or write to peripherals/memory controllers in response to DMA request signals.

The decode logic within the AHB decoder will need to be altered to take into account the address location of the DMA Controller and any other peripherals in the memory map. The base address of the DMA Controller needs to be defined on a system specific basis and the decode logic altered accordingly. If AMBA TIC methodology is to be used for integration testing then any change of address location for the DMA Controller block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the DMA Controller integration test program.

If multiple peripherals need to be connected to either of the AHB Master ports in the DMAC, then these AHB buses will need to have AHB Decoders connected to them. The default AHB decoder may be reused with suitable alterations to reflect the address maps of the respective AHB buses.

4.7 Interrupt Connection

The PrimeCell DMAC generates individual maskable active HIGH interrupts. Each interrupt can be enabled or disabled by changing its enable bit. Setting the enable bit HIGH enables the corresponding interrupt. The Error interrupts from all channels are ORed and output as the DMACINTERR interrupt. Similarly, the Terminal count interrupts from all channels are ORed and output as the DMACINTTC interrupt. Additionally, the interrupts are also output as a combined single interrupt, DMACINTR that is a combination of DMACINTERR and DMACINTTC. Either DMACINTERR and DMACINTTC or DMACINTR would be connected to the interrupt controller. The DMAC interrupts would then typically be routed to the nIRQ interrupt line to the ARM processor through the interrupt controller. The status of the individual interrupt sources can be read from status registers in the DMAC.

The DMACINTR would typically be routed to the nIRQ interrupt line of the ARM processor through an external Interrupt Controller.

4.8 Clocking

The ARM PrimeCell DMAC HDL contains a single clock source at the top level – HCLK. This lends itself very well to the ideal single scan chain test strategy.

4.9 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default DMA Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.10 Resets

The DMAC block has a single reset pin. The system bus reset signal HRESETn, resets all internal registers. HRESETn can be asserted asynchronously by the system reset controller but it should be deasserted synchronous to the rising edge of HCLK.

4.11 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.12 Synchronisers

Asynchronous DMA request signals from peripherals are synchronised by a series of two flip-flops inferred during synthesis from standard HDL templates. This synchronisation can be bypassed (on a per-peripheral basis) if the DMA requests are known to be synchronous to HCLK.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the DMAC block have been isolated to the DmacRqstSync sub-block. This allows the synthesis scripts to be modified so that these blocks could be synthesised separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler `set_prefer` command. Further synthesis of these sub-blocks can then be prevented using the `set_dont_touch` command. The rest of the DMAC design is then synthesised after placing a `set_dont_use` command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous DMA requests and eliminate the expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove 'X' propagation.

4.13 Endianness

The DMA Controller supports only 32-bit accesses on its AHB slave data buses. For a 32-bit system, the DMA Controller AHB slave interface is bi-endian to the extent that it can support 32-bit accesses from a little-endian master and a big-endian master. In systems with wider data buses, data routing will have to be done external to the DMAC. The endianness of the AHB Master ports of the DMAC is register-controlled.

4.14 Integration Testing

The DMA Controller integration test vector suite allows the integrator to verify that the DMA Controller has been connected into the target system correctly. These vectors test two classes of signals:

- a) AMBA signals (AHB signals connected to the AHB Slave port or the AHB Master ports of the DMA Controller signals)
- b) Non-AMBA Intra-Chip signals (DMA Controller signals connected to the Peripherals and Interrupt Controllers)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The integration vectors for the non-AMBA Intra-Chip signals are contained in the `DmacIntegTests.c` file in the `integration/bustest` directory. The integration vectors for the AHB slave port are contained in the `DmacResetTests.c` and `DmacRegisterTests.c` files in the same directory. The integration vectors for the two AHB Master ports are contained in the `DmacIntM2MTests.c` file in the `integration/bustest` directory. The file `Dmac.c` in the `integration/bustest` directory is used to selectively run some or all of these tests. As delivered, these vectors are intended to run correctly with the DMA Controller in stand-alone mode within the integration EASY world i.e. even before the DMA Controller has been integrated into the user system. After integration, the following points need to be noted:

- a) The integration vectors targeting the DMA Controller AHB slave port (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the DMA Controller AHB Master ports connectivity **NEED** to be modified by the integrator. The `DmacIntM2MTests.c` file has been embedded with NOTES listing the changes that the integrator would have to make in the test code.
- c) The integration vectors targeting the DMA Controller-to-Peripherals and DMA Controller-to-Interrupt Controller (Non-AMBA Intra-Chip signals) connectivity will **NEED** modification. The `IntegrationTest.c` file has been embedded with NOTES listing the changes that the integrator would have to make in the test code.

4.15 Production Testing

The DMA Controller is designed to be production tested using scan-insertion and ATPG.

4.15.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The DMA Controller design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used, the number of chip level scan chains required may vary. As delivered, the DMA Controller is synthesised with 1 scan-chain.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan insertion and ATPG approach is in cases where significant modifications have been made to the DMA Controller block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

4.16 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the DMA Controller world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

e.g. : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable.

e.g.: **setenv PERIPH Dmac**

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g. : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV**TEST_METH**

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

e.g. : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. : **setenv HDL_SOURCE vhdl**

setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g. : **setenv HDL_NETL vhdl**

setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. : **setenv TEST_ENV BUSTALK**
setenv TEST_ENV TICTALK

LIB_VHDL

The location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

e.g. : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

The location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

e.g. : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

The location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

e.g. : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

The location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

e.g.: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected - AHB, ASB or APB. Since the DMAC PL081 is an AHB peripheral, set this environment variable to 'AHB'. This environment variable is used by makefiles in the DMAC database to select settings for the Integration world.

e.g. : **setenv AMBA AHB**